



Java Abstract Classes and Interfaces

Section 1: Learn

What is an Abstract Class?

An **abstract class** is a class that **cannot be instantiated**, and is used to provide a base for subclasses.

- It can have **abstract methods** (without body)
- It can have **non-abstract methods** (with body)

Syntax:

```
abstract class Animal {  
    abstract void makeSound(); // abstract method  
  
    void sleep() {           // concrete method  
        System.out.println("Sleeping...");  
    }  
}
```

Characteristics of Abstract Class

- Can have both abstract and concrete methods
 - Can have constructors
 - Can have fields (variables)
 - Can be extended using **extends** keyword
 - Must be declared using **abstract** keyword
-



Real World Example of Abstract Class

```
abstract class Vehicle {  
    abstract void startEngine();  
  
    void fuelType() {  
        System.out.println("Petrol or Diesel");  
    }  
}  
  
class Car extends Vehicle {  
    void startEngine() {  
        System.out.println("Car engine started");  
    }  
}
```

What is an Interface?

An **interface** is a reference type in Java, similar to a class, that is a collection of **abstract methods**.

- Used to **achieve 100% abstraction**
- From Java 8, interfaces can also have **default** and **static** methods

Syntax:

```
interface Printable {  
    void print(); // abstract method  
}
```



Interface Methods

1. **Abstract** (default type)

```
void draw(); // implicitly abstract
```

2. **Default Method** (Java 8)

```
default void show() {  
    System.out.println("Default implementation");  
}
```

3. **Static Method** (Java 8)

```
static void greet() {  
    System.out.println("Hello from Interface");  
}
```

Interface vs Abstract Class

Feature	Interface	Abstract Class
Method Types	Abstract, default, static	Abstract & concrete
Fields	Only constants (public static final)	Can have variables
Constructors	Not allowed	Allowed
Inheritance	Implements	Extends
Multiple Inheritance	Supported	Not supported
Access Modifiers	Public only	All access modifiers allowed



When to Use Which?

- Use **interface** when:
 - You want to define a **contract** (behavior)
 - You need **multiple inheritance**
 - Use **abstract class** when:
 - You want to provide **partial implementation**
 - You want to **share code** between related classes
-

Real World Example

Abstract Class Example:

```
abstract class Appliance {  
    abstract void turnOn();  
    void plugIn() {  
        System.out.println("Plugged in");  
    }  
}  
  
class Fan extends Appliance {  
    void turnOn() {  
        System.out.println("Fan is on");  
    }  
}
```

Interface Example:

```
interface RemoteControl {  
    void turnOff();  
}
```



```
default void reset() {  
    System.out.println("Reset to factory settings");  
}  
  
class TV implements RemoteControl {  
    public void turnOff() {  
        System.out.println("TV turned off");  
    }  
}
```

Section 2: Practice

Create an Abstract Class

```
abstract class Shape {  
    abstract void draw();  
    void show() {  
        System.out.println("Shape is being displayed");  
    }  
}
```

Implementing an Interface

```
interface Vehicle {  
    void run();  
}
```



```
class Bike implements Vehicle {  
    public void run() {  
        System.out.println("Bike is running");  
    }  
}
```

Section 3: Know More (FAQs)

Can we create an object of abstract class?

No. Only subclasses can be instantiated.

Can a class implement multiple interfaces?

Yes. Java supports **multiple interface inheritance**.

Can interfaces have constructors?

No. Interfaces **cannot** have constructors.

Can I declare a method in interface with **private?**

Yes. From **Java 9**, **private** methods are allowed inside interfaces.

Is it necessary to override all interface methods?

Yes, unless the class is also declared **abstract**.